

Advanced ASIC Design

Exercise no 2: VCS and Verdi

Krzysztof Dziembała

2025-05-20

Contents

2	Project analysis - meta_harden	3
3	Simulation in text mode	4
4	Simulation in graphical mode	6
5	Coverage	7
6	UART transmitter	10

List of Figures

3.1	Output of simulation in text mode, together with script source. . . .	4
3.2	Output and code of simulation in text mode with additional <code>\$display</code> statements showing transmitted bits.	5
4.1	Waveform (from GUI) showing simulation of UART receiver.	6
5.1	Using <code>runsym.sh --cov</code> to generate and open coverage report in Verdi.	8
5.2	<code>uart_tb.uart_rx_i.uart_rx_ctl_i0.state</code> FSM transitions from coverage report.	9
5.3	Toggle coverage details for signals in <code>uart_rx_ctl</code>	9
6.1	SpyGlass lint results showing 3 violations in <code>lint/lint_rtl</code> goal, when run on initial version of the transmitter.	10
6.2	Lint results of the final transmitter implementation, showing only a single warning caused by unconnected <code>o_rx_in_i_clk</code> output of receiver, as required by the laboratory instruction.	12
6.3	Simulation of transmitter and receiver communicating without errors.	12
6.4	Coverage summary and line coverage.	13
6.5	Toggle coverage details.	13
6.6	FSM coverage details.	14
6.7	Condition coverage details showing <code>((state == IDLE) && i_send_data)</code> expression.	14
6.8	Branch coverage details showing that <code>i_send_data</code> and <code>baud_en</code> were never 1 at the same time when the transmitter was in IDLE state. . .	15

2. Project analysis - meta_harden

The `meta_harden` module contains a 2 stage synchronizer (`in` $\Rightarrow$ DFF $\Rightarrow$ DFF $\Rightarrow$ `out`) to prevent possible metastability caused by `i_rx_in` being an external input that is most likely not synchronised with the module's clock (`i_clk`).

3. Simulation in text mode

Ignoring the part responsible for removing previous results to always force full rebuild, the script executed two simple commands:

```
vcs -f filelist.f
./simv
```

The file list used by the command had the following content:

```
1 RTL/uart_tb.v
2 RTL/meta_harden.v
3 RTL/uart_baud_gen.v
4 RTL/uart_rx_ctl.v
5 RTL/uart_rx.v
```

```

$ runysimsh u x
Lab_2> $ runysimsh
1 #!/usr/bin/env bash
2 set -ex
3
4 ./clearsim.sh
5
6 vcs -f filelist.f
7 ./simv
8
$ clearsimsh u x
Lab_2> $ clearsimsh
1 #!/usr/bin/env bash
2 set -ex
3
4 # Remove previous results
5 if [ -d csrc ]; then
6     rm -rf csrc
7 fi
8 if [ -d simv.daidir ]; then
9     rm -rf simv.daidir
10 fi
11 if [ -f simv ]; then
12     rm simv
13 fi
14 if [ -f ucll.key ]; then
15     rm ucll.key
16 fi
$ filelist.f u x
Lab_2> $ filelist.f
1 RTL/uart_tb.v
2 RTL/meta_harden.v
3 RTL/uart_baud_gen.v
4 RTL/uart_rx_ctl.v
5 RTL/uart_rx.v
6
Starting vcs inline pass...
1 module and 0 UDP read.
recompiling module uart_tb
rm -f _cvarc*.so _csrc*.so pre_vcsobj.*.so share_vcsobj.*.so
if [ -x ../simv ]; then chmod a+x ../simv; fi
g++ -o ../simv -no-pie -rdynamic -Wl,-rpath=/simv.daidir -Wl,-rpath=/eda/synopsys/2023-12/RHELx86/VCS_2023.12/linux64/lib -L/eda/synopsys/2023-12/RHELx86/VCS_2023.12/linux64/lib -Wl,-rpath-link=/ -Wl,--no-as-needed obj/ancqw.d.o _432658_archive.1.so SIM.1.o rnapats_mop.o rnapats.o rnar.o rnar_nd.o rnar_llvm_0.1.o rnar_llvm_0.0.o -lvrsln -lerrorlnf -lsnpsnalloc -lvfs -lvcsnew -lsimprofile -luclina tline /eda/synopsys/2023-12/RHELx86/VCS_2023.12/linux64/lib/vcs_1ls.o -Wl,-whole-archive -lvcsucll -Wl,-no-whole-archive /eda/synopsys/2023-12/RHELx86/VCS_2023.12/linux64/lib/vcs_save_restore_new.o -ldl -lc -ln -lpthread -ldl
../simv up to date
CPU time: .178 seconds to compile + .167 seconds to elab + .214 seconds to link
+ ./simv
Info: [VCS_SAVE_RESTORE_INFO] ASLR (Address Space Layout Randomization) is detected on the machine. To enable $save functionality, ASLR will be switched off and simv re-executed.
Please use "-no-save" simv switch to avoid re-execution or "-suppress=ASLR_DETECTED_INFO" to suppress this message.
Chronologic VCS simulator copyright 1991-2023
Contains Synopsys proprietary information.
Compiler version V-2023.12_Full64; Runtime version V-2023.12_Full64; May 14 12:32 2025
Data received on UART interface:
Data received (hex, ASCII) = 65 at time: 82280000
Data received on UART interface:
Data received (hex, ASCII) = 66 at time: 169228000
Data received on UART interface:
Data received (hex, ASCII) = 6f at time: 256160000
Data received on UART interface:
Data received (hex, ASCII) = 44 at time: 342560000
Data received on UART interface:
Data received (hex, ASCII) = 20 at time: 429500000
Data received on UART interface:
Data received (hex, ASCII) = 6c at time: 516440000
Data received on UART interface:
Data received (hex, ASCII) = 66 at time: 603380000
Data received on UART interface:
Data received (hex, ASCII) = 65 at time: 689780000
Data received (hex, ASCII) = 57 at time: 776720000
Sfinish called from file "RTL/uart_tb.v", line 82.
Sfinish at simulation time 890680000
VCS Simulation Report
Time: 890680000 ps
CPU Time: 0.180 seconds; Data structure size: 0.0Mb
Wed May 14 12:32:51 2025
student@labrv26: ~/Documents/kdz/ASIC/Lab_2$

```

Figure 3.1: Output of simulation in text mode, together with script source.

Simulation output is presented on Fig. 3.1.

In the next step of this exercise, `$display` statements, shown below, have been added to the code responsible for setting `rx_in` and produced output visible on Fig. 3.2.

```
diff --git a/Lab_2/RTL/uart_tb.v b/Lab_2/RTL/uart_tb.v
index 034f2ae..00e276d 100644
--- a/Lab_2/RTL/uart_tb.v
+++ b/Lab_2/RTL/uart_tb.v
@@ -89,6 +89,11 @@ module uart_tb
    bit_num = bit_num + 1;
    rx_in = bits_to_send[bit_num];
    if (bit_num >= BIT_TO_SEND_NUM) rx_in = 1'b1;
+   if (bit_num < BIT_TO_SEND_NUM) begin
+

```

```

+         if ((bit_num-1) % 10 == 0) $display("Start Bit, at time: %t",
↪ $time);
+         else if ((bit_num-1) % 10 == 9) $display("Stop Bit, at time %t",
↪ $time);
+         else $display("Data send = %0d, at time: %t", rx_in, $time);
+         end
    end

always @ (posedge data_rdy) //data monitor

```

```

15 .o_rx_data (data_out),
16 .o_rx_data_rdy (data_rdy),
17 .o_frm_err ( )
18 );
19
20 initial //Data to send preparation
21 begin
22     bytes_to_send = { 8'h57,
23                       8'h65,
24                       8'h6c,
25                       8'h6c,
26                       8'h20,
27                       8'h44,
28                       8'h6f,
29                       8'h6e,
30                       8'h65 };
31
32     bits_to_send[0] = 1'b1;
33
34     for (i = 0; i < BYTE_TO_SEND_NUM; i=i+1)
35     begin
36         bits_to_send[(i*10+1):10] = add_start_and_stop_bit(bytes_to_send[i]);
37     end
38 end
39
40 initial //Main Initil
41 begin
42     clk = 1'b1;
43     rst = 1'b0;
44     rx_in = 1'b1;
45     bit_num = 0;
46
47     #4000
48     rst = 1'b1;
49
50     #((BIT_TO_SEND_NUM + 10) * BAUD_RATE_T + 10000)
51     $finish;
52 end
53
54 always #((CLK_T/2) clk = ~clk; //clock
55
56 always #((BAUD_RATE_T) //data sender
57 begin
58     bit_num = bit_num + 1;
59     rx_in = bits_to_send[bit_num];
60     if (bit_num >= BIT_TO_SEND_NUM) rx_in = 1'b1;
61     if (bit_num < BIT_TO_SEND_NUM) begin
62         if ((bit_num-1) % 10 == 0) $display("Start Bit, at time: %t", $time);
63         else if ((bit_num-1) % 10 == 9) $display("Stop Bit, at time %t", $time);
64         else $display("Data send = %0d, at time: %t", rx_in, $time);
65     end
66 end
67
68 always @ (posedge data_rdy) //data monitor
69 begin
70     $display("Data received on UART interface:");
71     $display("Data received (hex, ASCII) = %h at time: %t", data_out, $time);
72 end
73
74 function [9:0] add_start_and_stop_bit;
75 input [7:0] data;
76 begin
77     add_start_and_stop_bit = {1'b1,data,1'b0};
78 end
79 endfunction
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111

```

```

Data received (hex, ASCII) = 44 at time: 342560000
Stop Bit, at time 347200000
Start Bit, at time: 355800000
Data send = 0, at time: 364560000
Data send = 0, at time: 373240000
Data send = 0, at time: 381920000
Data send = 0, at time: 390600000
Data send = 1, at time: 399280000
Data send = 0, at time: 407960000
Data send = 0, at time: 416640000
Data send = 0, at time: 425320000
Data received on UART interface:
Data received (hex, ASCII) = 20 at time: 429500000
Stop Bit, at time 434000000
Start Bit, at time: 442680000
Data send = 0, at time: 451360000
Data send = 0, at time: 460040000
Data send = 1, at time: 468720000
Data send = 1, at time: 477400000
Data send = 0, at time: 486080000
Data send = 1, at time: 494760000
Data send = 1, at time: 503440000
Data send = 0, at time: 512120000
Data received on UART interface:
Data received (hex, ASCII) = 6c at time: 516440000
Stop Bit, at time 520800000
Start Bit, at time: 529480000
Data send = 0, at time: 538160000
Data send = 0, at time: 546840000
Data send = 1, at time: 555520000
Data send = 1, at time: 564200000
Data send = 0, at time: 572880000
Data send = 1, at time: 581560000
Data send = 1, at time: 590240000
Data send = 0, at time: 598920000
Data received on UART interface:
Data received (hex, ASCII) = 6c at time: 603380000
Stop Bit, at time 607600000
Start Bit, at time: 616280000
Data send = 1, at time: 624960000
Data send = 0, at time: 633640000
Data send = 1, at time: 642320000
Data send = 0, at time: 651000000
Data send = 0, at time: 659680000
Data send = 1, at time: 668360000
Data send = 1, at time: 677040000
Data send = 0, at time: 685720000
Data received on UART interface:
Data received (hex, ASCII) = 65 at time: 689780000
Stop Bit, at time 694400000
Start Bit, at time: 703080000
Data send = 1, at time: 711760000
Data send = 1, at time: 720440000
Data send = 1, at time: 729120000
Data send = 0, at time: 737800000
Data send = 1, at time: 746480000
Data send = 0, at time: 755160000
Data send = 1, at time: 763840000
Data send = 0, at time: 772520000
Data received on UART interface:
Data received (hex, ASCII) = 57 at time: 776720000
Stop Bit, at time 781200000
$finish called from file "RTL/uart_tb.v", line 82.
$finish at simulation time 890680000
V.C.S. Simulation Report
Time: 890680000 ps
CPU Time: 0.190 seconds; Data structure size: 0.0Mb
Wed May 14 13:00:16 2025

```

Figure 3.2: Output and code of simulation in text mode with additional \$display statements showing transmitted bits.

According to the simulation, the receiver works correctly and receives all the data without any errors. Figures 3.1 and 3.2 show the receiver returning a sequence 0x65, 0x6e, 0x6f, 0x44, 0x20, 0x6c, 0x6c, 0x65, 0x57 (ASCII string “Well Done” in reverse), with Fig. 3.2 displaying the bits being sent (e.g. <start>11101010<end> for 0x57, because UART is LSB first). Data ready is reported before stop bit is received and verified, but this is documented, therefore I assume this was the intention.

4. Simulation in graphical mode

To run the simulation with GUI, the script was modified to accept `--gui` flag and use different `vcs` and `./simv` invocations:

```
1  #!/usr/bin/env bash
2  set -ex
3
4  ./clearsim.sh
5
6  if [ $1 == "--gui" ]; then
7      vcs -kdb -debug_access+all -f filelist.f
8      ./simv -gui
9  else
10     vcs -f filelist.f
11     ./simv
12 fi
```

When running the simulation with GUI, the waveform (Fig. 4.1) shows the receiver working correctly and performing like it did in text-only simulation.

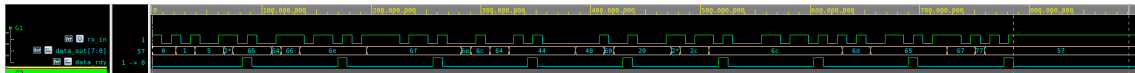


Figure 4.1: Waveform (from GUI) showing simulation of UART receiver.

5. Coverage

The final version of the script, enabling coverage collection and displaying results in GUI when run with `--cov` is presented below:

```
1  #!/usr/bin/env bash
2  set -ex
3
4  ./clearsim.sh
5
6  if ! command -v vcs; then
7      echo "vsc command not found. Did you forget to run alias
8          ↪ add-synopsys-FEV-all?"
9      exit 1
10  fi
11
12  if [ $1 == "--gui" ]; then
13      vcs -kdb -debug_access+all -f filelist.f
14      ./simv -gui
15  elif [ $1 == "--cov" ]; then
16      vcs -cm line+tgl+cond+fsm+branch -f filelist.f
17      ./simv -cm line+tgl+cond+fsm+branch
18      urg -dir simv.vdb
19      verdi -cov -covdir simv.vdb
20  else
21      vcs -f filelist.f
22      ./simv
23  fi
```

For completeness, the source of `clearsim.sh` is as follows:

```
1  #!/usr/bin/env bash
2  set -e
3
4  # Remove previous results
5  rm -rf csrc simv.daidir verdiLog ucli.key simv.vdb urgReport vdCovLog
6  rm -f simv ucli/key inter.fsdb* .inter.fsdb.* .vcs_checkpoint_* novas*
7  ↪ verdi_config_file .fsm.sch.verilog.xml cm.log
```

The window with coverage report opened by the script is shown on Fig. 5.1.

Fig. 5.2 presents transitions of `state` FSM in the receiver module. During the simulation transitions from `DATA` and `START` states to `IDLE` have not occurred. The transition from `DATA` to `IDLE` can occur only if reset is asserted (`i_rst` goes low) when receiver is receiving data. Transition `START` $\Rightarrow$ `IDLE` can be caused by reset or a glitch that makes `i_rx_in_clk` go low just for a short moment, but is not a proper start bit. The only reset in the simulation happened at the beginning and no glitch while idle was simulated, so neither of those two transitions could happen.

The tool shows many different types of coverage:

- **line**, which shows which statements were or were not executed,

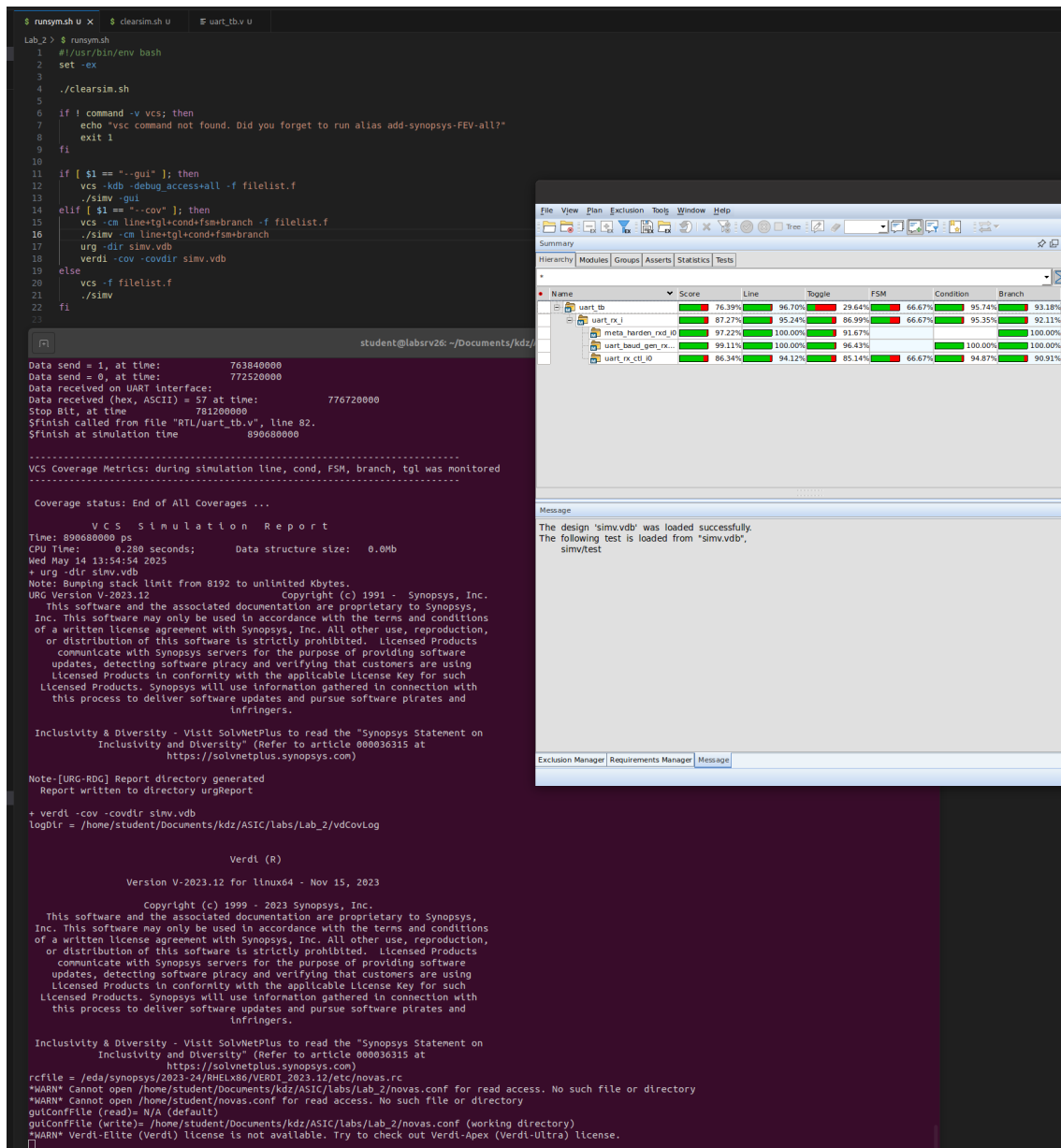


Figure 5.1: Using `runsym.sh --cov` to generate and open coverage report in Verdi.

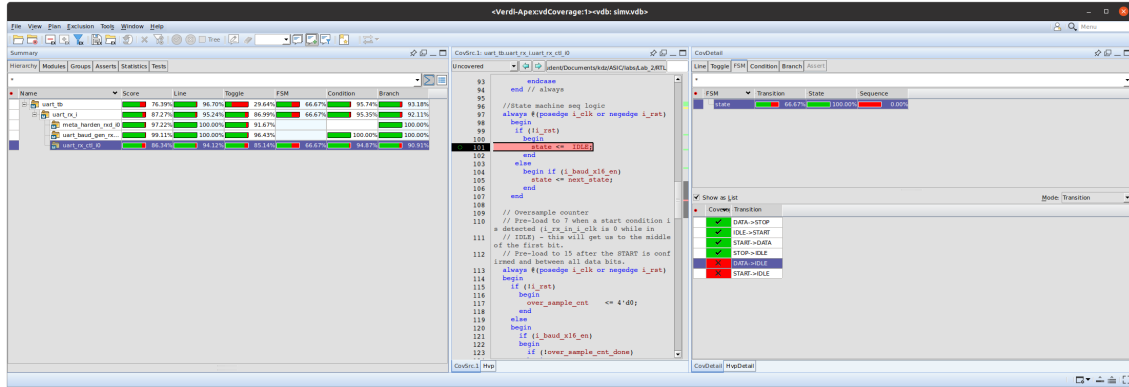
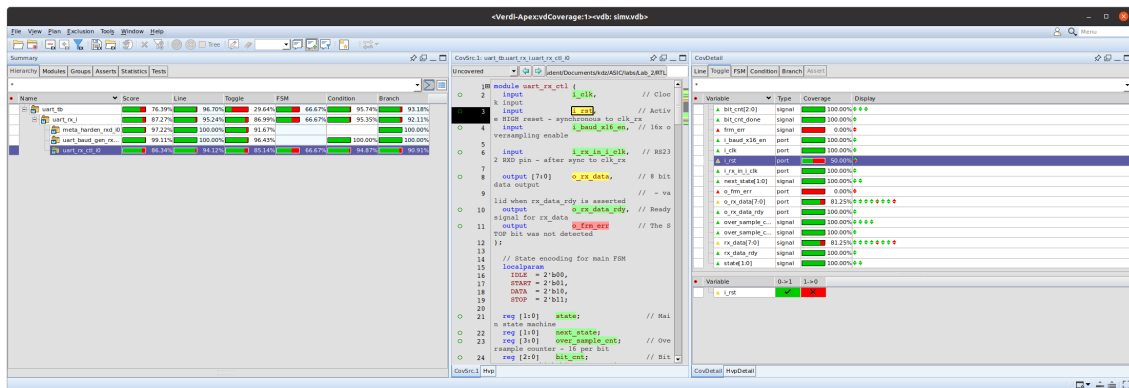


Figure 5.2: `uart_tb.uart_rx_i.uart_rx_ctl_i0.state` FSM transitions from coverage report.

- **toggle**, presented on Fig. 5.3, which shows which variables, down to a single bit, have changed and in what way (e.g. `i_rst` only toggled from 0 to 1),
- **FSM**, shown on Fig. 5.2, which shows states that were visited, observed transitions and sequences,
- **condition**, which shows whether conditional expressions (and their broken down sub-expressions) were ever true or false and in which combinations,
- **branch**, which shows which branches of `if` and `case` statements were or were not hit during simulation,
- **assert**, which was not available, but should show SystemVerilog `assert` and `cover` statements.



6. UART transmitter

To make implementation of transmitter easier, the existing `uart_baud_gen` module was modified to accept a parameter (`OVERSAMPLE_MUL`) specifying oversampling rate, instead of using hardcoded $\times 16$ multiplier. Along with this change, net names were changed to remove `_x16`.

As the transmitter module was being written, the code was checked using SpyGlass. The first complete version had violated 3 rules, as shown on Fig. 6.1.

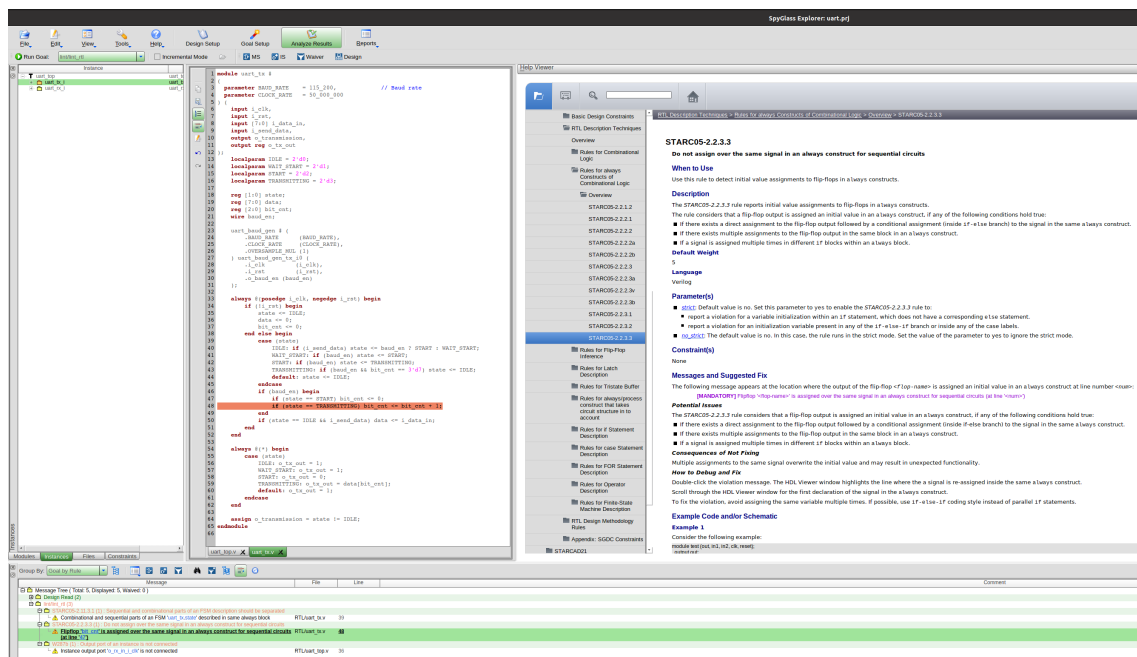


Figure 6.1: SpyGlass lint results showing 3 violations in `lint/lint_rtl` goal, when run on initial version of the transmitter.

Final version of the transmitter, presented below, did not violate any rules, as shown on Fig. 6.2.

```

1 module uart_tx #
2 (
3     parameter BAUD_RATE      = 115_200,           // Baud rate
4     parameter CLOCK_RATE     = 50_000_000
5 ) (
6     input i_clk,
7     input i_rst,
8     input [7:0] i_data_in,
9     input i_send_data,
10    output o_transmission,
11    output reg o_tx_out
12 );

```

```

13     localparam IDLE = 2'd0;
14     localparam WAIT_START = 2'd1;
15     localparam START = 2'd2;
16     localparam TRANSMITTING = 2'd3;
17
18     reg [1:0] state;
19     reg [1:0] next_state;
20     reg [7:0] data;
21     reg [2:0] bit_cnt;
22     wire baud_en;
23
24     uart_baud_gen # (
25         .BAUD_RATE      (BAUD_RATE),
26         .CLOCK_RATE     (CLOCK_RATE),
27         .OVERSAMPLE_MUL (1)
28     ) uart_baud_gen_tx_i0 (
29         .i_clk           (i_clk),
30         .i_rst           (i_rst),
31         .o_baud_en       (baud_en)
32     );
33
34     always @(posedge i_clk, negedge i_rst) begin
35         if (!i_rst) begin
36             state <= IDLE;
37             data <= 0;
38             bit_cnt <= 0;
39         end else begin
40             state <= next_state;
41             if (baud_en) begin
42                 if (state == START) bit_cnt <= 0;
43                 else if (state == TRANSMITTING) bit_cnt <= bit_cnt + 1;
44             end
45             if (state == IDLE && i_send_data) data <= i_data_in;
46         end
47     end
48
49     always @(*) begin
50         case (state)
51             IDLE: next_state = i_send_data ? (baud_en ? START : WAIT_START) : IDLE;
52             WAIT_START: next_state = baud_en ? START : WAIT_START;
53             START: next_state = baud_en ? TRANSMITTING : START;
54             TRANSMITTING: next_state = (baud_en && bit_cnt == 3'd7) ? IDLE :
                    ↪ TRANSMITTING;
55             default: next_state = IDLE;
56         endcase
57     end
58
59     always @(*) begin
60         case (state)
61             IDLE: o_tx_out = 1;
62             WAIT_START: o_tx_out = 1;
63             START: o_tx_out = 0;
64             TRANSMITTING: o_tx_out = data[bit_cnt];
65             default: o_tx_out = 1;
66         endcase
67     end
68

```

```

69     assign o_transmission = state != IDLE;
70 endmodule

```

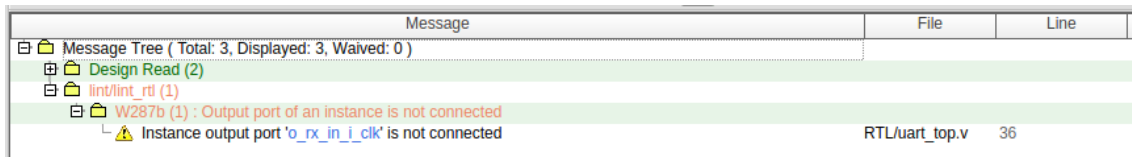


Figure 6.2: Lint results of the final transmitter implementation, showing only a single warning caused by unconnected `o_rx_in_i_clk` output of receiver, as required by the laboratory instruction.

Simulation results of fully working transmitter-receiver pair are shown on Fig. 6.3.

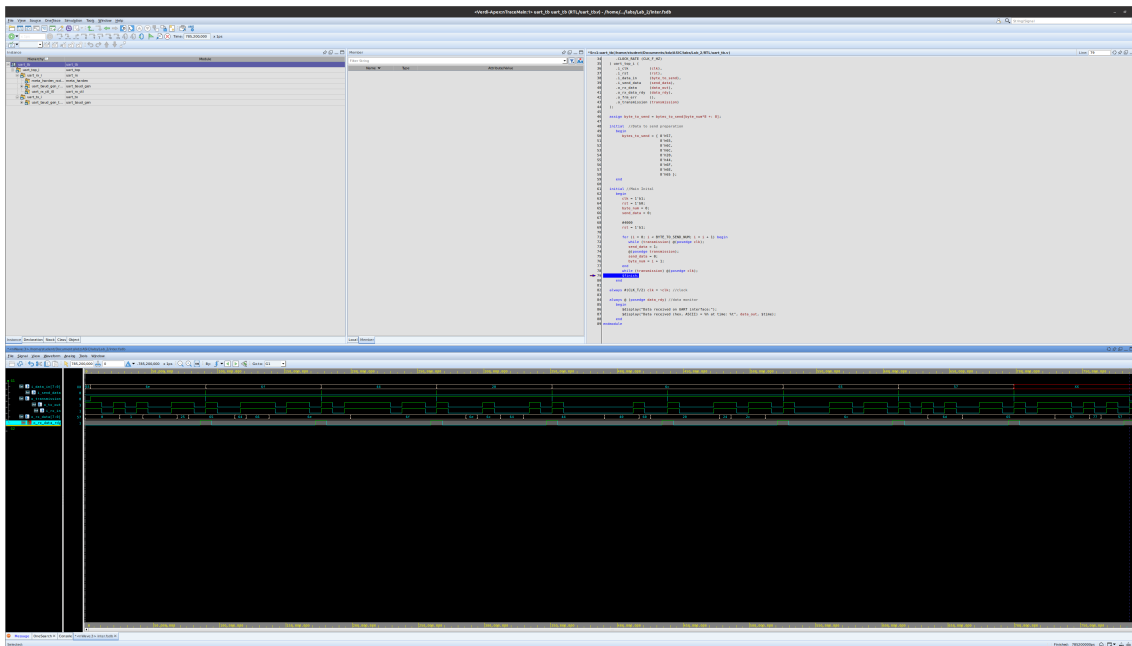


Figure 6.3: Simulation of transmitter and receiver communicating without errors.

Coverage results (Figures 6.4, 6.5, 6.6, 6.7 and 6.8) show that most of the code was covered during test. Interesting exceptions are:

1. `state` not going from `START` or `WAIT_START` to `IDLE` (see Fig. 6.6), because `i_rst` does not go low during test, as shown on Fig. 6.5,
2. `state` not going from `IDLE` to `START` (see Fig. 6.6), because `baud_en` is never high when `i_send_data` is high and the FSM is in `IDLE` state (Fig. 6.7 and 6.8),
3. default statement in `case (state)` that sets `o_tx_out` not being covered (see Fig. 6.4 and 6.8), which is interesting, since `case (state)` for `next_state` has it covered.

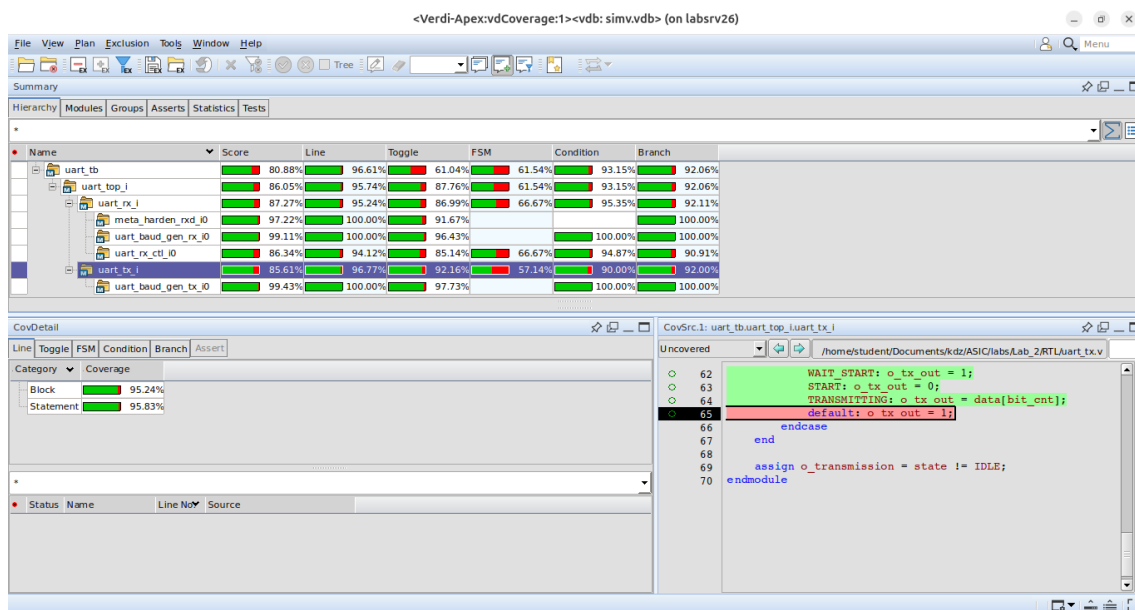


Figure 6.4: Coverage summary and line coverage.

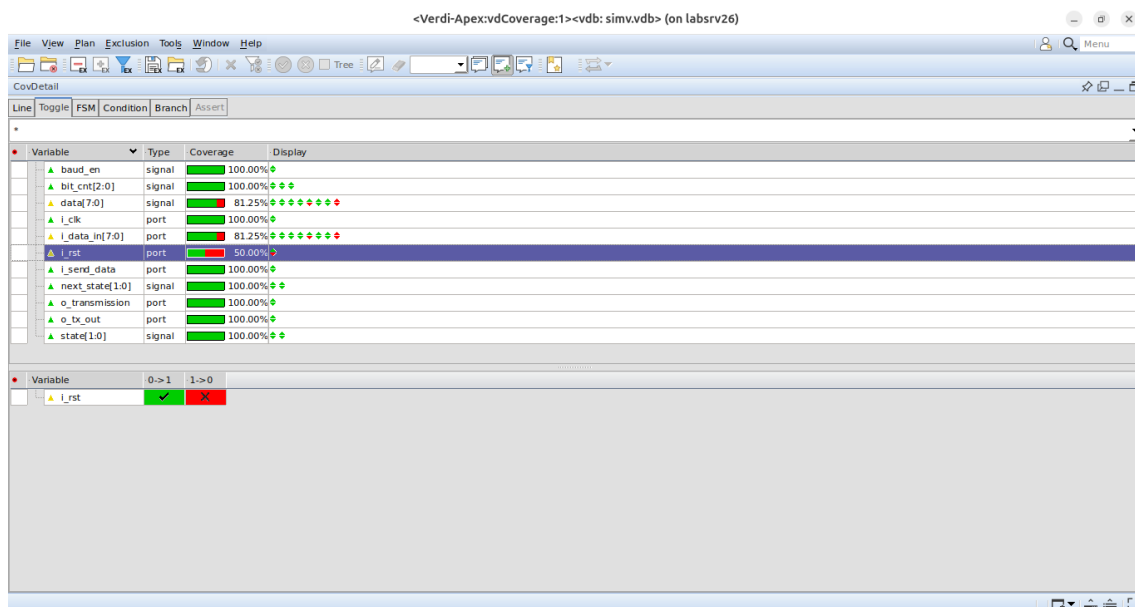


Figure 6.5: Toggle coverage details.

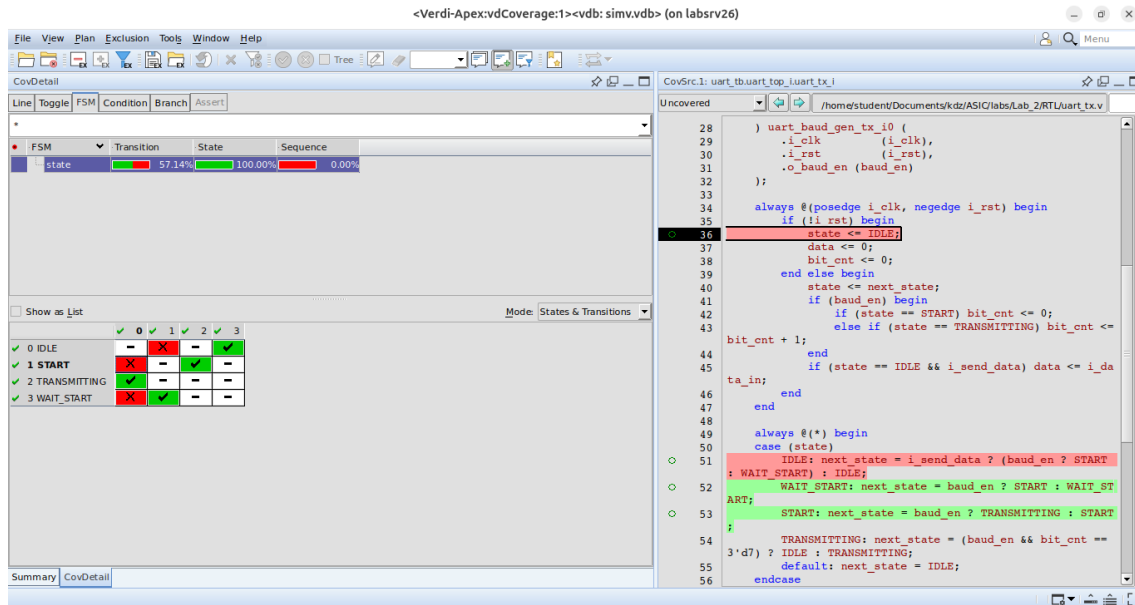


Figure 6.6: FSM coverage details.

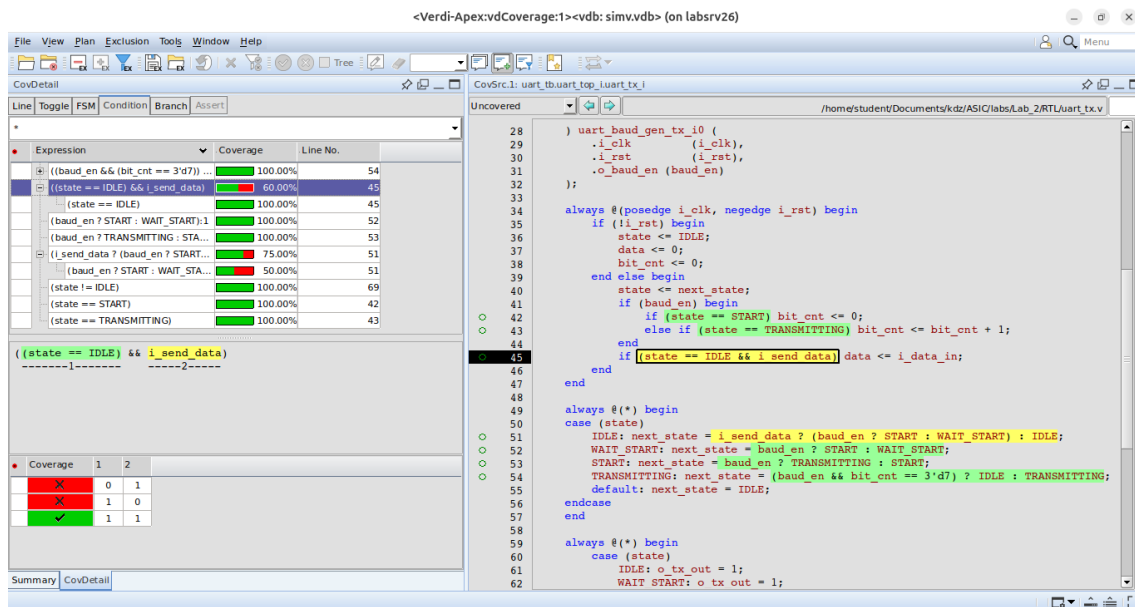


Figure 6.7: Condition coverage details showing ((state == IDLE) && i_send_data) expression.

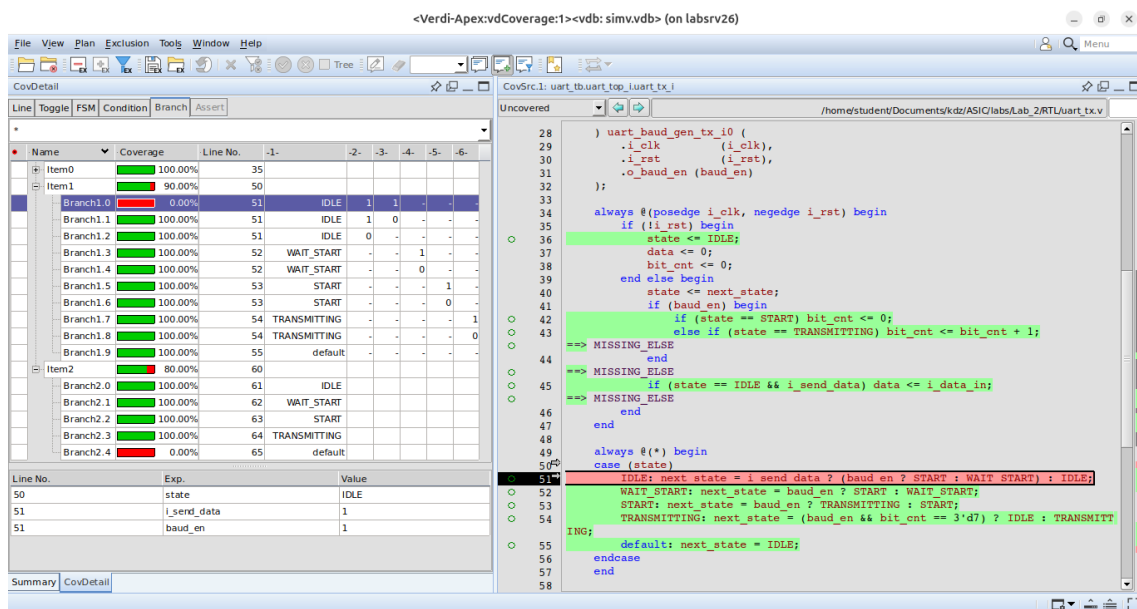


Figure 6.8: Branch coverage details showing that `i_send_data` and `baud_en` were never 1 at the same time when the transmitter was in IDLE state.